

Cahier de TP

« Messagerie distribuée avec Kafka »

C# – .NET

Pré-requis :

- Bonne connexion Internet
- Système d'exploitation : Windows
- IDE Recommandés : Visual Studio
- Docker, Git
- JDK21+

Images utilisées :

- *apache/kafka:4.2.0*
- *redpandadata/console:v3.5.3*
- *confluentinc/cp-schema-registry :8.2.0*
- *postgres:15.1*
- *dpage/pgadmin4*
- *docker.elastic.co/elasticsearch/elasticsearch:7.7.1*
- *docker.elastic.co/kibana/kibana:7.7.1*

Table des matières

Atelier 1: Le cluster kafka.....	4
1.1 Premier démarrage et logs.....	4
1.2 Utilisation des utilitaires	4
1.3 Outils graphiques	4
Ateliers 2 : Producer API	5
Atelier 3 : Consumer API.....	6
3.0 Mise en place d'une base Postgres.....	6
3.1 Implémentation	6
3.2 Tests.....	6
3.3 EOS Semantics (Optionnel).....	7
Atelier 4. Schema Registry et sérialisation Avro	8
4.1 Ajout de Confluent Schema Registry et outils Avro	8
4.2 Producteur de message.....	8
4.3 Consommateur de message.....	8
4.4 Mise à jour du schéma	8
4.4.1 Evolution du schéma compatible	8
4.4.2 Evolution du schéma incompatible	9
Atelier 5. Kafka Connect	10
5.1 Installation du plugin ElasticSearch dans kafka connect	10
5.2 Démarrage de la stack	10
5.3 Configuration du connecteur.....	10
5.3 Améliorations	11
Atelier 6 : Garanties Kafka	12
6.1. Transaction et ISOLATION_LEVEL.....	12
6.1.1 Producer	12
6.1.2 Consommateur	12
6.2 Sémantique Exactly Once (Consume – Transform – Produce).....	12
6.2.1 Principe	12
6.2.2 Implémentation	12
6.2.3 Preuve de la garantie Exactly Once	13
6.3 Stockage et rétention.....	13
Atelier 7 : Streamiz	15
7.1 Opérateurs stateless.....	15
7.2 Opérateurs stateful	15
Atelier 8 : ksqlDB	Erreur ! Signet non défini.
8.1 Getting started.....	Erreur ! Signet non défini.
Ateliers 9: Sécurité.....	20

9.0 Séparation des échanges réseaux	20
9.1 Accès via SSL au cluster	20
9.2 Authentification avec SASL/PLAIN.....	20
9.2.1 Authentification inter-broker.....	20
9.2.2 Authentification client.....	20
9.3 ACL	21
9.4 Quotas	21
Atelier 10 : Monitoring JMX, Prometheus, Grafana	22

Atelier 1: Le cluster kafka

1.1 Premier démarrage et logs

Visualiser le fichier *docker-compose.yml*

Il permet de démarrer un cluster 3 nœuds ainsi qu'un container utile pour l'administration du cluster

Démarrer cette stack avec :

```
docker compose up -d
```

Observer les logs de démarrages d'un nœud

```
docker logs -f kafka-0
```

1.2 Utilisation des utilitaires

Ouvrir un bash sur un des nœuds

```
docker exec -it kafka-0 bash
```

Créer un topic *testing* avec 5 partitions et 2 répliques :

```
/opt/kafka/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --  
replication-factor 2 --partitions 5 --topic testing
```

Lister les topics du cluster

Accéder à la description détaillée du topic

Démarrer un producteur de message

```
/opt/kafka/bin/kafka-console-producer.sh -- bootstrap-server localhost:9092 --  
topic testing --property "parse.key=true" --property "key.separator="
```

Saisir quelques messages

Consommer les messages depuis le début

```
/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --  
topic testing --from-beginning
```

Dans une autre fenêtre, lister les groupes de consommateurs et accéder au détail du groupe de consommateur en lecture sur le topic *testing*

1.3 Outils graphiques

Parcourir l'UI de Redpanda Console :

Redpanda Console : <http://localhost:8080>

Ateliers 2 : Producer API

Créer un nouveau projet *Producer* de type *ApplicationConsole* avec les packages *Confluent.Kafka*

Récupérer les sources fournis. La classe principale *Program.cs* qui prend en arguments :

- `nbThreads` : Un nombre de threads/task
- `nbMessages` : Un nombre de messages
- `sendMode` : Le mode d'envoi : 0 pour `Fire_And_Forget`, 1 pour `Synchrone`, 2 pour `Asynchrone`

L'application instancie `nbThreads ProducerThread` et leur demande d'envoyer `nbMessages` dans le mode d'envoi spécifié en ligne de commande.

Lorsque toutes les tâches sont terminées. Elle affiche le temps d'exécution

Le projet est constitué des classes suivantes :

- Une classe *ProducerThread* que vous devez implémenter en particulier la construction du producteur Kafka et les méthodes d'envoi de messages.
Les messages sont constitués d'une clé au format string (`courier.id`) et d'une valeur au format JSON (classe `Position`)
- Un sérialiseur Custom capable de sérialiser en Json n'importe quelle classe.
- Le package `model` contient les classes modélisant les données
 - *Position* : Une position en latitude, longitude
 - *Courier* : Un coursier associé à une position
 - *SendMode* : Une énumération des modes d'envoi

Implémenter les 3 modes d'envoi de la classe *ProducerThread*.

Tester l'envoi de messages dans les différents modes dans l'IDE

Via les commandes utilitaires ou l'interface d'admin, vérifier la création du topic et le contenu des messages

Construire un exécutable

Supprimer le topic et le recréer avec un nombre de *partitions=5* et un *replication-factor=3*

Envoyer 100 000 de messages, par exemple :

```
bin/Debug/KafkaProducer.exe 100 1000 1
```

Atelier 3 : Consumer API

3.0 Mise en place d'une base Postgres

Démarrer un serveur postgres et sa console d'administration

```
docker compose -f postgres-docker-compose.yml up -d
```

Accéder à localhost:81 et se logger avec *admin@admin.com/admin*

Enregistre un serveur avec comme paramètres de connexion :

- host : *consumer-postgresql*
- user : *postgres*
- password : *postgres*

Créer une base consumer et y exécuter le script de création de table fourni : *create-table.sql*

3.1 Implémentation

Créer un nouveau projet avec les packages *Confluent.Kafka* et *Npgsql*

Récupérer le code fourni

Le projet est composé de :

- Une classe principale *KafkaConsumerApplication* qui prend en arguments :
 - *nbThreads* : Un nombre de threads
L'application instancie *nbThreads* *KafkaConsumerThread* et leur demande de consommer les messages sous le même nombre de groupe. Le programme s'arrête avec un *Ctrl+C*.
- *KafkaConsumerThread* lors de la réception de message insère en base dans la table *coursier* :
 - L'id du coursier et l'offset du message
- Le package model contient les classes modélisant les données
 - *Position* : Une position en latitude, longitude
 - *Courier* : Un coursier associé à une position

Compléter la boucle de réception des messages

Pour tester la réception, vous pouvez utiliser le programme précédent et le lancer afin qu'il exécute de nombreux messages.

Positionner des handler écoutant la réaffectation de partitions

3.2 Tests

Une fois le programme mis au point, construire

Effectuer plusieurs tests, entre chaque démarrage effacer les offsets du groupe et nettoyer la base de données

Tester qu'aucun message n'est perdu :

- Démarrer le programme avec 3 threads
- Visualiser la répartition des partitions
- Arrêter puis redémarrer avec la même configuration

Tester la réaffectation de partitions :

- Démarrer 2 fois l'application : 1 avec 2 threads l'autre avec 3 threads
- Visualiser la répartition des partitions
- Arrêter et redémarrer un processus pendant la consommation

Test un redémarrage broker

- Arrêter un broker :

`docker stop kafka-0`

- Redémarrer le, Dans le répertoire du fichier *docker-compose*

`docker compose up -d`

- Essayer de vérifier le traitement de tous les messages. (Il se peut qu'il y ait des doublons de traitement)

3.3 EOS Semantics (Optionnel)

En implémentant un *PartitionsAssignedHandler*, faire en sorte que les offsets de redémarrage d'un consommateur soit relu à partir de la base de données plutôt que Kafka.

Atelier 4. Schema Registry et sérialisation Avro

4.1 Ajout de Confluent Schema Registry et outils Avro

Visualiser le fichier *docker-compose.yml* fourni

Démarrer la stack

Accéder à *localhost:8081/subjects*

Visualiser dans la Redpanda Console le schema registry

4.2 Producteur de message

Créer un nouveau projet *ProducerAvro* avec les packages *Confluent.Kafka*,
Confluent.SchemaRegistry.Serdes.Avro,

Récupérer le schéma Avro fourni : *Coursier.avsc*

Installer globalement le package *Apache.Avro.Tools* :

```
dotnet tool install --global Apache.Avro.Tools
```

Dans le répertoire du fichier Schema, exécuter :

```
avrogen -s Coursier.avsc .
```

Reprendre les classes du projet *producer* sans les classes du modèle ni le serialiseur

Dans la classe principale, enregistrer le schéma dans le serveur registry :

Dans le producteur de message modifier la classe *KafkaProducerThread* afin

- Qu'elle compile
- Qu'elle utilise un sérialiseur de valeur de type *AvroSerializer*

Modifier le nom du topic d'envoi en *position-avro* et tester la production de message.

Accéder à <http://localhost:8081/subjects>

Puis à accéder à <http://localhost:8081/schemas/>

Vérifier que redpanda puissent lire les messages.

4.3 Consommateur de message

Créer un nouveau projet *ConsumerAvro* avec les packages *Confluent.Kafka*,
Confluent.SchemaRegistry.Serdes.Avro,

Reprendre les sources du projet *KafkaConsumer* sans les classes du modèle ni le désérialiseur

Modifier le code fin d'utiliser la classe *GenericRecord*

Modifier le désérialiseur du thread de consommation.

4.4 Mise à jour du schéma

4.4.1 Evolution du schéma compatible

Mettre à jour le schéma en ajoutant les champs optionnels : *firstName* dans la structure *Coursier*

```
{  
  "name": "first_name",
```



```
"type": "string",  
"default": "undefined"  
},
```

Fixer les problèmes de compilation

Relancer le programme de production et visualiser la nouvelle version du schéma dans le registre

Consommer les messages sans modifications du programme consommateur

4.4.2 Evolution du schéma incompatible

Mettre à jour le schéma en ajoutant un champ obligatoire : *vehicle_id* dans la structure Coursier

```
{  
"name": "vehicle_id",  
"type": "int"  
},
```

Fixer les problèmes de compilation

Relancer le programme de production et visualiser l'exception au moment de l'enregistrement du nouveau schéma.

Visualiser les nouveaux messages publiés dans le topic

Atelier 5. Kafka Connect

Objectifs : Déverser le topic position dans un index ElasticSearch

5.1 Installation du plugin ElasticSearch dans kafka connect

Visualiser le fichier Dockerfile fourni et construire une image *my-kafka-connect-with-elasticsearch* via :

```
docker build -t my-kafka-connect-with-elasticsearch .
```

5.2 Démarrage de la stack

Démarrer la nouvelle stack qui ajoute l'image précédemment construite, ElasticSearch et Kibana

```
docker-compose up -d
```

Vérifier l'installation du plugin via :

<http://localhost:8083/connector-plugins>

Se connecter à kibana localhost:5601 et dans la DevConsole exécuter :

```
PUT /position
{
  "mappings": {
    "properties": {
      "@timestamp": {
        "type": "date",
        "format": "epoch_millis"
      }
    }
  }
}
```

La commande crée un index elasticsearch *position* avec pour l'instant un seul champ *@timestamp*

5.3 Configuration du connecteur

Définir un connecteur *elasticsearch-sink* qui déverse le topic position dans l'index position :

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "elasticsearch-sink",
  "config": {
    "connector.class":
"io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",
    "tasks.max": "1",
    "topics": "position",
    "topic.index.map": "position:position_index",
    "connection.url": "http://elasticsearch:9200",
    "type.name": "log",
    "key.ignore": "true",
    "schema.ignore": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false"
  }
}'
```

Vous pouvez vérifier la bonne installation du connecteur :

- Via les logs de KafkaConnect
- Via `http://localhost:8083/connectors` : Liste des connecteurs actifs
- Via l'interface de RedPanda

Alimenter le topic position

Vous pouvez visualiser les effets du connecteur

- Via ElasticSearch : `http://localhost:9200/position/_search`
- Via Kibana : <http://localhost:5601>

5.3 Améliorations

- Améliorer le fichier de configuration afin d'introduire que le timestamp Kafka soit renseigné dans le champ `@timestamp` de l'index position d'ElasticSearch
- Déverser le topic *position-avro* dans un index *position-avro*

Atelier 6 : Garanties Kafka

6.1. Transaction et ISOLATION_LEVEL

6.1.1 Producer

Modifier le code du *Producer* afin d'englober plusieurs envois de messages dans une transaction.

Commiter tous les 10 envois

Lancer ensuite le programme avec comme arguments 1 105 0

Ce qui signifie que 1 threads envoie 105 messages, les 100 premiers messages font partie de 10 transactions validées. Les 5 derniers messages ne sont pas validés.

Visualiser les messages dans une console d'administration.

Noter le nombre et les offsets, Visualiser le flag transactionnel

6.1.2 Consommateur

Vérifier que le consommateur ne consomme que 100 messages

6.2 Sémantique *Exactly Once* (Consume – Transform – Produce)

Objectif : Mettre en œuvre le pattern *Consume-Transform-Produce* avec la garantie *Exactly Once*. Prouver cette garantie en simulant un crash.

6.2.1 Principe

Le programme lit les messages depuis le topic *position*, calcule la distance de chaque coursier par rapport à un point d'origine (latitude=0, longitude=0) et produit le résultat dans un topic de sortie : *position-output*.

L'ensemble (lecture + transformation + écriture + commit des offsets) est englobé dans une **transaction atomique** garantissant qu'un message est traité **exactement une fois**.

6.2.2 Implémentation

Créer un nouveau projet *ExactlyOnceProcessor* avec le package Confluent.Kafka.

Récupérer le code fourni.

Configuration du Producer transactionnel :

```
EnableIdempotence = true
```

```
TransactionalId = "exactly-once-processor"
```

Configuration du Consumer :

```
GroupId = "exactly-once-group"
EnableAutoCommit = false
IsolationLevel = IsolationLevel.ReadCommitted
```

Boucle de traitement :

Initialisation de la transaction

Récupération d'un lot de messages.

Transformation de chaque message du lot

Envoi atomique de tous les messages transformés

6.2.3 Preuve de la garantie Exactly Once

Préparation :

- Supprimer les topics *position* et *position-output* et les recréer
- Produire exactement **100 messages** dans le topic position avec le Producer de l'atelier 2

Test sans crash :

1. Lancer *ExactlyOnceProcessor* avec 100 messages
2. Compter les messages dans position-output → **100 messages attendus**

Test avec crash et restart :

1. Lancer ExactlyOnceProcessor avec 100 messages
2. L'interrompre avec Ctrl+C **avant** qu'il ait fini (par exemple après ~50 messages)
3. Lire les messages committés avec kafka console-consumer :

```
docker exec -it kafka-0 bash
/opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic position-output \
  --from-beginning \
  --consumer-property isolation.level=read_committed
```
4. Relancer ExactlyOnceProcessor avec 100 messages
5. Vérifier que le consommateur configuré en read-committed n'a que 100 messages

6.3 Stockage et rétention

Ouvrir un bash sur un des noeuds

```
docker exec -it kafka-0 bash
```

Visualiser les répertoires de logs sur les brokers :

```
kafka-log-dirs.sh --bootstrap-server localhost:9092 --describe
```

Visualiser les segments du topic position et apprécier la taille

Pour le topic position modifier le ***segment.bytes*** à 1Mo

Diminuer le ***retention.bytes***

Produire de nombreux messages afin de voir des segments disparaître

Atelier 7 : Streamiz

Objectifs :

Écrire une application Stream qui prend en entrée le topic *position-avro*

7.1 Opérateurs stateless

Créer un projet *PositionStream* et installer le package

Streamiz.Kafka.Net.SchemaRegistrySerdesAvro

Reprendre le schéma Avro initial du producteur Avro et générer les classes :

avrogen -s Coursier.avsc .

Dans un premier temps, transformer les informations de la position d'un coursier en arrondissant la longitude et la latitude à 1 valeur entière.

Tester, regarder les timestamp des messages

Supprimer le topic de sortie

Compléter le stream en inversant les clés et Valeurs (La position du coursier devient la clé, l'id du coursier la valeur)

Tester

Utiliser l'opérateur *split/branch* pour créer 2 topics de sortie un contenant les positions dont latitude est supérieure à 45.0 et l'autre le reste

Tester

7.2 Opérateurs stateful

7.2.1 Agrégation Count

Sur les branches créées en 7.1, effectuer une agrégation de type Count() : compter le nombre de coursiers ayant passé à une position donnée.

Tester.

Modifier en utilisant une fenêtre temporelle de 1 seconde.

Tester.

7.2.2 Coursiers actuellement présents par position

Cette étape permet de savoir « combien de coursiers actuellement présents » dans une zone.

- Construire une KTable<coursierId, position> qui mémorise la dernière position connue de chaque coursier (via GroupByKey + Reduce qui garde la valeur la plus récente).
- Faire un GroupBy sur la position, puis un Aggregate avec adder et subtractor :

adder (+1) : un coursier arrive sur la position

subtract (-1) : un coursier quitte sa position précédente

Publier le compteur dans le topic coursiers-par-position.

Atelier 8 : ksqlDB

Objectifs

- Découvrir l'interface CLI de ksqlDB
- Créer des streams et des tables à partir du topic `position` produit aux ateliers précédents
- Écrire des requêtes dérivées pour filtrer et agréger les positions des coursiers
- Comprendre la différence entre Push Query (temps réel) et Pull Query (snapshot)

Contexte

Dans cet atelier, ksqlDB consomme directement le topic `position` produit par JMeter (atelier 2). Aucun code Java n'est nécessaire — tout se fait en SQL depuis la CLI ou via l'API REST.

8.0 Démarrage de la stack

Un fichier `docker-compose-ksqldb.yml` vous est fourni. Il étend la stack existante avec :

- **ksqlDB Server** (port 8088)
- **ksqlDB CLI** (client interactif)

```
docker compose -f docker-compose-ksqldb.yml up -d
```

Vérifier que le serveur est opérationnel :

```
curl http://localhost:8088/info
```

8.1 Getting Started

Connexion à la CLI

```
docker exec -it ksqldb-cli ksql http://ksqldb-server:8088
```

Explorer l'environnement :

```
SHOW TOPICS;  
SHOW STREAMS;  
SHOW TABLES;
```

Inspecter le topic `position` pour voir les messages existants :

```
PRINT 'position' FROM BEGINNING LIMIT 5;
```

Observer la structure d'un message : `coursierId`, `latitude`, `longitude`, `timestamp`.

Créer le stream de base

Créer un stream `positions` mappé sur le topic `position` :

```
CREATE STREAM positions (  
  courierId VARCHAR,  
  latitude DOUBLE,  
  longitude DOUBLE  
) WITH (  
  kafka_topic = 'position',  
  value_format = 'JSON'  
);
```

Vérifier la création :

```
DESCRIBE positions;
```


Exécuter une première push query pour voir les messages en temps réel. **Dans un autre terminal**, démarrer la production de message dans le topic position

```
SELECT courierId, latitude, longitude
FROM positions
EMIT CHANGES
LIMIT 10;
```

Observer les messages défiler. Appuyer sur `Ctrl+C` pour arrêter.

8.2 Transformations et filtres

Stream dérivé — positions dans Paris

Créer un stream ne contenant que les positions dans la zone parisienne (latitude entre 48.8 et 48.95, longitude entre 2.2 et 2.5) :

```
CREATE STREAM positions_paris AS
  SELECT courierId,
         latitude,
         longitude
  FROM positions
  WHERE latitude BETWEEN 48.80 AND 48.95
         AND longitude BETWEEN 2.20 AND 2.50
  EMIT CHANGES;
```

Vérifier que le topic correspondant a été créé :

```
SHOW TOPICS;
```

Constater que ksqldb a créé automatiquement le topic POSITIONS_PARIS.

Exécuter une push query sur ce stream filtré :

```
SELECT * FROM positions_paris EMIT CHANGES LIMIT 5;
```

Table — dernière position connue par coursier

Créer une table qui maintient en temps réel la **dernière position connue** de chaque coursier :

```
CREATE TABLE last_position AS
  SELECT courierId,
         LATEST_BY_OFFSET(latitude) AS last_lat,
         LATEST_BY_OFFSET(longitude) AS last_lon,
         COUNT(*) AS nb_messages
  FROM positions
  GROUP BY courierId
  EMIT CHANGES;
```

La table est mise à jour à chaque nouveau message reçu pour un coursier donné.

Agrégation par zone — coursiers par secteur

Arrondir les coordonnées à 1 décimale pour créer des secteurs géographiques, et compter combien de coursiers différents ont été vus dans chaque secteur :

```
CREATE TABLE positions_par_secteur AS
```

```

SELECT CAST(ROUND(last_lat, 1) AS VARCHAR) + '_' +
       CAST(ROUND(last_lon, 1) AS VARCHAR) AS secteur,
COUNT(*) AS nb_coursiers,
COLLECT_LIST(courierId) AS coursiers
FROM last_position
GROUP BY CAST(ROUND(last_lat, 1) AS VARCHAR) + '_' +
         CAST(ROUND(last_lon, 1) AS VARCHAR)
EMIT CHANGES;

```

Faire une sélection sur la table

8.3 Requêtes

Push Query — suivi en temps réel

Démarrer la production de messages

Dans la CLI, suivre en temps réel les mises à jour de la table `last_position` :

```

SELECT courierId, last_lat, last_lon, nb_messages
FROM last_position
EMIT CHANGES;

```

Observer les lignes se mettre à jour au fil des messages produits.

Filtrer sur un seul coursier (remplacer `courier-0` par un ID visible dans les logs) :

```

SELECT *
FROM last_position
WHERE courierId = 'courier-0'
EMIT CHANGES;

```

Pull Query — snapshot à l'instant T

Une fois JMeter arrêté, interroger la table pour obtenir l'état courant :

```

-- Dernière position de tous les coursiers
SELECT courierId, last_lat, last_lon, nb_messages
FROM last_position;

```

```

-- Coursier ayant envoyé le plus de messages
SELECT courierId, nb_messages
FROM last_position
WHERE nb_messages > 50;

```

Pull Query vs Push Query : la pull query retourne un snapshot et se termine immédiatement. La push query reste ouverte et pousse les mises à jour en continu.

Push Query avec fenêtre temporelle

Compter les messages par coursier sur des fenêtres glissantes de 1 minute :

```

SELECT courierId,
       WINDOWSTART AS debut,
       WINDOWEND   AS fin,
       COUNT(*)    AS nb
FROM positions
WINDOW TUMBLING (SIZE 1 MINUTE)
GROUP BY courierId
EMIT CHANGES;

```

Relancer la production et observer les compteurs se réinitialiser à chaque nouvelle fenêtre.

8.4 Pour aller plus loin (optionnel)

Requête de proximité avec GEO_DISTANCE

Créer une table des coursiers proches de la Place de la République (48.8672, 2.3628) :

```
CREATE TABLE coursiers_proche_republique AS
  SELECT courierId,
         last_lat,
         last_lon,
         ROUND(GEO_DISTANCE(last_lat, last_lon, 48.8672, 2.3628), 2) AS distance_km
  FROM last_position
  WHERE GEO_DISTANCE(last_lat, last_lon, 48.8672, 2.3628) <= 5.0
EMIT CHANGES;
```

Pull query pour voir les coursiers actuellement proches :

```
SELECT * FROM coursiers_proche_republique;
```

Explorer les objets créés

```
DESCRIBE EXTENDED last_position;
SHOW QUERIES;
EXPLAIN <query_id>;
```

DESCRIBE EXTENDED affiche les statistiques d'exécution de la requête sous-jacente.

SHOW QUERIES liste toutes les requêtes persistantes en cours d'exécution.

Nettoyage

```
DROP TABLE coursiers_proche_republique DELETE TOPIC;
DROP TABLE positions_par_secteur DELETE TOPIC;
DROP TABLE last_position DELETE TOPIC;
DROP STREAM positions_paris DELETE TOPIC;
DROP STREAM positions;
```

Ateliers 9: Sécurité

9.0 Séparation des échanges réseaux

Le fichier `docker-compose` sépare déjà sur des ports différents la communication contrôleurs, brokers et client externe.

Re-visualiser la configuration des listeners

9.1 Accès via SSL au cluster

Redémarrer le cluster en utilisant la nouvelle version de `docker-compose`, le port `EXTERNAL` utilise dorénavant SSL

Récupérer le répertoire SSL qui contient un *keystore* et un *trustore* self-signed pour localhost

Modifier `docker-compose.yml` afin que le répertoire *ssl/mount* soit correctement monté sur chaque broker.

Démarrer le cluster et vérifier le certificat produit par le serveur :

```
openssl s_client -debug -connect localhost:19092 -tls1_3
```

Modifier les propriétés du client `Producer` afin qu'il utilise le protocole SSL pour communiquer

```
SecurityProtocol = SecurityProtocol.Ssl,
```

```
SslCaLocation = "/path/to/ca-cert.pem",
```

Vérifier la production de message

9.2 Authentification avec SASL/PLAIN

9.2.1 Authentification inter-broker

Mettre au point un fichier `kafka_server_jaas.conf` définissant 2 utilisateurs admin et alice et indiquant que le serveur utilise l'identité admin comme suit :

```
KafkaServer {  
  org.apache.kafka.common.security.plain.PlainLoginModule required  
  username="admin"  
  password="admin-secret"  
  user_admin="admin-secret"  
  user_alice="alice-secret";  
};
```

Récupérer la nouvelle version de `docker-compose` et vérifier le montage de volume vers votre fichier `kafka_server_jaas.conf`

Redémarrer le cluster et vérifier son bon démarrage

9.2.2 Authentification client

Configurer le projet *Producer* pour qu'il s'authentifie avec l'utilisateur *alice*

```
SecurityProtocol = SecurityProtocol.SaslSsl, // Protocole sécurisé SASL_SSL
```

```
SaslMechanism = SaslMechanism.Plain, // Mécanisme d'authentification PLAIN
```

```
SaslUsername = "alice", // Nom d'utilisateur
```

```
SaslPassword = "alice-secret", // Mot de passe
```

```
SslCaLocation = "<path-to>\\ca-cert.pem",
```

Tester l'envoi de message, vérifier la bonne configuration du *KafkaProducer* et la production de message

9.3 ACL

Visualiser la nouvelle configuration des brokers dans le nouveau *docker-compose.yml*

Démarrer le cluster.

Essayer de produire des messages

Utiliser le script `setup-acls.sh` pour positionner des ACLs sur les topics

Docker exec `-it kafka-0 bash`

```
/tmp/setup-acls.sh
```

Vérifier via Redpanda Console que les ACLs sont bien visibles

Essayer de produire des messages

9.4 Quotas

Utiliser le script `/tmp/setup-quotas.sh` pour positionner des quotas sur la production.

Vérifier les quotas positionnés dans Redpanda Console

Modifier le code du Producer afin d'afficher un message lorsque la production est freinée.

```
.SetStatisticsHandler((_, json) =>
{
    try
    {
        using var doc = JsonDocument.Parse(json);
        var root = doc.RootElement;
        if (root.TryGetProperty("brokers", out var brokers))
        {
            foreach (var broker in brokers.EnumerateObject())
            {
                var b = broker.Value;
                if (b.TryGetProperty("throttle", out var throttle))
                {
                    var cnt =
throttle.GetProperty("cnt").GetInt64();
                    var sum =
throttle.GetProperty("sum").GetInt64();
                    if (cnt > 0)
                    {
                        Console.WriteLine($"[THROTTLE] Broker
{broker.Name} : {cnt} requêtes throttlés, temps total = {sum} ms");
                    }
                }
            }
        }
    }
    catch { /* ignore parsing errors */ }
})
```

Démarrer la production avec beaucoup de threads et visualiser les messages sur la console

Atelier 10 : Monitoring JMX, Prometheus, Grafana

Exécuter les producteurs et les consommateurs pendant les opérations d'administration

Dans un premier temps, démarrer une JConsole et visualiser les Mbeans des brokers, consommateurs et producteurs

Visualiser le nouveau docker-compose.yml qui :

- ☐ Attache l'agent *jmx_prometheus_javaagent-0.20.0.jar* aux processus Java des brokers
- ☐ Intègre et configure les services Prometheus et Grafana

Produire et consommer

Vérifier la production de métriques par chaque broker :

http://localhost:7071

Vérifier la récupération des métriques dans Prometheus :

http://localhost:9091

Connecter-vous à Grafana

- ☐ *http://localhost:3000* avec admin/admin

Vérifier la datasource Prometheus

Visualiser les tableaux de bord

Les métriques des brokers devraient s'afficher